

INDIVIDUAL PROJECT REPORT

ON

“CampusSphere: Admin Consoles, Gate Scanner PWA and Deployment for a Unified College Event and Activity-Points Platform”

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of
Technology in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Ankit Yadav

(Roll No.: _____)

Member, Group Project CampusSphere

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

**ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR**

CERTIFICATE

This is to certify that the individual project report entitled “**CampusSphere: Admin Consoles, Gate Scanner PWA and Deployment for a Unified College Event and Activity-Points Platform**”, submitted by **Ankit Yadav (Roll No.: _____)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by her/him as a member of the project group (Govind Jangid, Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak) at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his continuous guidance, encouragement and valuable suggestions throughout the project. His insistence on clear requirements, a weekly plan and honest review helped me understand how a large software system is planned and delivered by a team.

I sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for the academic support and the environment provided for this work. I am also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

I would like to acknowledge the support of my teammates, **Govind Jangid, Garvita Jain, Ayushi Garg and Anshika Pathak**, with whom I worked on **CampusSphere**. Our discussions on interfaces between the frontend, backend and database sections, the joint review of the specification and the shared planning of the 14-week schedule greatly enriched my understanding of team-based software development. I especially thank **Govind Jangid**, our Team Leader, for coordinating the weekly meetings and the integration of all five sections.

I also thank the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events shaped the requirements of the platform.

Finally, I extend my heartfelt gratitude to my family and friends for their constant support, patience and motivation throughout the course of this project.

Ankit Yadav
(Roll No.: _____)
Computer Science & Engineering

ABSTRACT

This report presents my individual contribution to the group project **CampusSphere**, a unified web platform for college events, club treasuries, verified attendance and AICTE activity points at **Arya College of Engineering & I.T., Jaipur**. The work was divided among five students into two frontend and deployment sections (Govind Jangid and myself) and three backend and database sections (Garvita Jain, Ayushi Garg and Anshika Pathak).

My section covers the administrator-facing frontend and the deployment. On the frontend I am responsible for the Club Admin console with the event creation wizard, the venue conflict interface, the Super Admin approval queue and finance and audit console, the camera-based gate scanner progressive web app with its green and red result screens and alarm, the “You Said, We Did” Kanban board, the transcript screens and the Event Copilot panel. On the deployment side I am responsible for Dockerising the application, configuring NGINX with TLS and rate limiting, the CI/CD pipeline, the staging and production deployment, monitoring and the runbook.

The report explains the technologies used (React, TypeScript, form validation, browser camera APIs, drag-and-drop, charts, Docker, Docker Compose, NGINX and GitHub Actions), the design of each module, my 14-week plan, the dependencies on my teammates, the test plan, and the challenges anticipated with their planned solutions.

This is a design and planning report. The targets quoted, such as scan response time and deployment reliability, are goals to be measured during testing and the pilot, not measured results. The work provides practical experience in building operational consoles, device-camera applications and a repeatable deployment pipeline.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION1

- 1.1 Project Overview1
- 1.2 Team and Work Division1
- 1.3 My Role and Responsibilities1
- 1.4 Objectives of My Section2
- 1.5 Project Duration and Schedule2
- 1.6 Tools and Collaboration3

CHAPTER 2 TECHNOLOGIES AND CONCEPTS4

- 2.1 React and TypeScript4
- 2.2 Forms and Validation4
- 2.3 TanStack Query for Tables and Pagination4
- 2.4 Role-Based Routing4
- 2.5 Browser Camera and Media APIs4
- 2.6 Web Audio and Vibration4
- 2.7 Progressive Web Apps5
- 2.8 WebSocket Client5
- 2.9 Charts and Data Tables5
- 2.10 Drag and Drop for the Kanban Board5
- 2.11 Docker and Docker Compose5
- 2.12 NGINX and TLS5
- 2.13 CI/CD with GitHub Actions5
- 2.14 Monitoring and Load Testing6

CHAPTER 3 INDUSTRY APPLICATIONS AND RELEVANCE7

- 3.1 Admin Consoles and Box-Office Tools7
- 3.2 Mobile Ticket Scanners7
- 3.3 Feedback Boards and Public Roadmaps7
- 3.4 Containerisation and DevOps7
- 3.5 Financial Dashboards and Auditability7
- 3.6 Future Scope7

CHAPTER 4 MY WORK: DESIGN AND IMPLEMENTATION APPROACH8

- 4.1 Administrator Application Architecture8
- 4.2 Club Admin Console8
- 4.3 Event Creation Wizard8
- 4.4 Venue Availability and Conflict Interface8
- 4.5 Super Admin Approval Queue9
- 4.6 Finance and Audit Console9
- 4.7 Gate Scanner Progressive Web App9
- 4.8 “You Said, We Did” Kanban Board10
- 4.9 Transcript and Sign-Off Screens11
- 4.10 Event Copilot Panel11
- 4.11 Deployment Architecture11
- 4.12 Containers and Compose12
- 4.13 NGINX, TLS and Rate Limiting12
- 4.14 CI/CD Pipeline12
- 4.15 Environment, Secrets and Backups13
- 4.16 Monitoring, Rollback and Runbook13
- 4.17 Testing Approach13
- 4.18 Anticipated Challenges and Planned Solutions13
- 4.19 Integration with Teammates14

CHAPTER 5 14-WEEK WORK PLAN15

- 5.1 My Week-by-Week Plan15

5.2 Team Overview and Dependencies15

5.3 Reporting and Review16

CHAPTER 6 LEARNING OUTCOMES AND SKILLS17

6.1 Technical Skills17

6.2 Professional Skills17

6.3 Teamwork17

6.4 Problem Solving and Debugging17

6.5 Version Control and Communication17

CHAPTER 7 CONCLUSION18

REFERENCES19

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The group project **CampusSphere** is a unified platform for college events, club treasuries, verified attendance and AICTE activity points at Arya College of Engineering & I.T., Jaipur. It replaces chat-group announcements, form-based registration, personal UPI collection, paper attendance and static certificates with one authenticated system for Students, Club Admins and the Super Admin (Dean/HOD). The platform includes a directory of fifteen clubs with dedicated treasuries, an event approval workflow, venue booking with a 30-minute buffer, atomic seat reservation, Razorpay payments credited to club ledgers, a rolling 30-second QR pass, a camera-based gate scanner, attendance-gated feedback, SHA-256-sealed certificates and an automated activity-points ledger.

The project was carried out by five students over 14 weeks under the guidance of **Er. Ram Babu Buri**. This individual report describes the part assigned to me: the frontend of the administrator side (Club Admin and Super Admin consoles, the gate scanner PWA and the Kanban board) and the deployment of the whole system (Docker, NGINX, TLS and CI/CD).

1.2 Team and Work Division

The work was divided into five sections. Two members, Govind Jangid and Ankit Yadav, took the frontend and deployment. Three members, Garvita Jain, Ayushi Garg and Anshika Pathak, took the backend and the database. Table 1.1 shows the division.

Table 1.1: Division of work among the team

Member	Section	Responsibility in brief
Govind Jangid (Team Leader)	Frontend (student experience)	Student screens, rolling QR pass, checkout, certificates and points UI, AI feed, 3D twin; integration lead
Ankit Yadav	Frontend (admin consoles) and deployment	Club and Super Admin consoles, gate scanner PWA, Kanban, Docker, NGINX, CI/CD
Garvita Jain	Backend	Authentication, RBAC, clubs, events, venue buffer engine, audit, security
Ayushi Garg	Backend	Registration, capacity lock, payments, ledger, QR token service, certificate PDF
Anshika Pathak	Database and backend	Schemas, indexes, seeding, transactions, attendance, feedback, seal and points

1.3 My Role and Responsibilities

My role has two parts. As the **frontend developer for the administrator side**, I was assigned the following modules:

- the application shell for administrators, route guards and the shared API client together with Govind Jangid;
- the Club Admin console: club profile, executive leads, registrations roster and a read-only treasury view with ledger entries;
- the event creation wizard with validation, draft saving and the Event Copilot panel;
- the venue availability and conflict interface with alternative venue suggestions;
- the Super Admin approval queue with approve and reject modal and the finance and audit console;
- the gate scanner progressive web app with camera QR reading, instant result screens, alarm, live counter and manual entry fallback;
- the “You Said, We Did” Kanban board for suggestions;
- the transcript download and Dean sign-off screens.
- As the **deployment owner**, I prepare Dockerfiles and the Compose stack, NGINX with TLS and rate limiting, the CI/CD pipeline, environment and secret handling, health checks, backups and the staging and production deployments.

1.4 Objectives of My Section

- To give Club Admins a fast and clear console that replaces spreadsheets and paper forms for organising events.
- To make the gate scanner usable by a volunteer in a crowded entrance: one glance must show whether to let a person in.
- To give the Super Admin full visibility of approvals and club finances with no ability to edit ledgers.
- To make deployment a repeatable, one-command process that anyone in the team can run.
- To protect the production system with HTTPS, rate limiting, secret handling and health monitoring.

- To keep admin screens consistent with the student application through a shared design system.

1.5 Project Duration and Schedule

The project is planned over 14 weeks and follows the ten-phase roadmap: foundation and authentication (Weeks 1-2), clubs (Week 3), events and venues (Weeks 4-5), registration and rolling QR (Week 6), payments (Week 7), gate scanner (Week 8), feedback (Week 9), certificates and points (Week 10), AI suite (Week 11), 3D twin and deployment (Week 12), integration testing (Week 13) and pilot with documentation (Week 14). My frontend work follows the backend APIs by about one week, and my deployment work is concentrated in Weeks 12 to 14, with only the development environment and the CI skeleton in Week 1. My week-by-week plan is in Chapter 5.

1.6 Tools and Collaboration

I use Visual Studio Code, Node.js, Git and GitHub with feature branches and pull requests, Docker Desktop, Postman, browser developer tools with mobile emulation and a physical phone for camera testing, GitHub Actions for CI/CD and k6 for load testing. The team communicates through a weekly meeting and a shared issue board.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter explains the technologies that I use for the administrator frontend and for deployment, and the concepts behind them.

2.1 React and TypeScript

The administrator application is a React 19 single-page application written in TypeScript. Typed interfaces for events, venues, registrations, ledger entries and suggestions are shared with the student application so that both sides agree on the API contract. Components such as data tables, steppers and modals are reused across consoles.

2.2 Forms and Validation

The event wizard has many fields with rules, such as a start time before the end time, a team size minimum not above the maximum and a price that is required only for paid events. I use React Hook Form with a schema validator (Zod) so that validation rules are declared once, errors are shown next to the field and the same schema also gives the TypeScript type. The browser validation gives quick feedback, but the backend remains the final authority.

2.3 TanStack Query for Tables and Pagination

Admin screens show long lists such as registrations or ledger rows. TanStack Query provides caching, background refresh, pagination and mutations with invalidation, so that approving an event updates the queue and the dashboard without manual refreshing.

2.4 Role-Based Routing

React Router with a guard component reads the role from the authentication context and redirects visitors who are not CLUB_ADMIN or SUPER_ADMIN. For club-scoped pages the guard also checks that the club identifier is in the user's `clubs` claim. This improves usability only; the real protection is the authorisation check in the backend.

2.5 Browser Camera and Media APIs

The scanner uses `navigator.mediaDevices.getUserMedia` to open the rear camera, draws video frames to a canvas and decodes QR codes using the `BarcodeDetector` API where available or a JavaScript decoder library (jsQR) as a fallback. Camera access needs HTTPS and a user permission prompt, which is why TLS is part of my deployment work.

2.6 Web Audio and Vibration

The duplicate-scan alarm uses the Web Audio API to generate a loud tone without an audio file, and the Vibration API on supported phones. A short, distinct tone confirms a valid scan, so a volunteer can work by sound and colour without reading text.

2.7 Progressive Web Apps

A web app manifest and a service worker make the scanner installable and keep its application shell available when the network is weak. Scans always need the server for validation, but the interface starts instantly and shows a clear offline state.

2.8 WebSocket Client

A WebSocket subscription receives live attendee counts and gate events from the server. The client reconnects with an increasing delay if the connection drops and re-fetches the current count after reconnecting so that the displayed number is never stale.

2.9 Charts and Data Tables

The finance console uses a charting library (Recharts) for revenue per club and a time series of daily collections, and a table component with sorting, filtering and CSV export for ledger entries.

2.10 Drag and Drop for the Kanban Board

The suggestion board uses a drag-and-drop library (dnd-kit) with keyboard support. A drop calls the status API and the card moves optimistically; if the server rejects the transition the card returns to its column and a message appears.

2.11 Docker and Docker Compose

Docker packages an application with its dependencies in an image that runs the same way everywhere. Docker Compose describes several containers (NGINX with the frontend, API, MongoDB replica set and optional Redis) in one file with a private network, volumes and health checks. A multi-stage build compiles the frontend in a Node image and copies only static files into a small NGINX image.

2.12 NGINX and TLS

NGINX serves the static frontend, proxies `/api` and WebSocket traffic to the API container, compresses responses, sets security headers and applies rate limiting (100 requests per minute

in general, 5 per minute for authentication). TLS certificates are obtained from Let's Encrypt and renewed automatically. TLS 1.3 is enabled.

2.13 CI/CD with GitHub Actions

Continuous integration runs on every pull request: install, lint, type check, unit tests, build and container build. Continuous delivery deploys the main branch to staging automatically and to production after approval. Pipelines are described in YAML files kept with the code.

2.14 Monitoring and Load Testing

Health endpoints and structured JSON logs make the system observable. k6 scripts simulate flash registrations and scanning at the gate to check the performance targets (event listing under 200 ms, gate validation under 400 ms).

CHAPTER 3

INDUSTRY APPLICATIONS AND RELEVANCE

3.1 Admin Consoles and Box-Office Tools

Ticketing companies provide organiser dashboards for creating events, monitoring sales and exporting attendee lists. The CampusSphere consoles follow the same pattern but add an approval stage and a club treasury under college audit.

3.2 Mobile Ticket Scanners

Venue staff use handheld or phone-based scanners that give a fast green or red result. Their design principles, such as large colour areas, instant feedback and a clear duplicate warning, apply directly to the gate scanner.

3.3 Feedback Boards and Public Roadmaps

Product teams use Kanban-style boards and public roadmaps to show users that their suggestions are being handled. The “You Said, We Did” board applies the idea to student events.

3.4 Containerisation and DevOps

Most modern software is delivered in containers through automated pipelines. Reproducible builds reduce the “it works on my machine” problem, and automated tests before deployment reduce the risk of breaking a live event.

3.5 Financial Dashboards and Auditability

Finance teams need dashboards that show totals, splits and the trail behind each number. The Super Admin console shows club-by-club revenue with a link from every figure to its ledger entries and payment identifier.

3.6 Future Scope

The administrator side can be extended with bulk operations, scheduled reports, a native scanner application, role delegation for event volunteers, blue-green deployment and automatic scaling when the platform is used by many colleges.

CHAPTER 4

MY WORK: DESIGN AND IMPLEMENTATION APPROACH

This chapter describes the design of the modules assigned to me. It is based on the project specification suite and on the schedule in Chapter 5.

4.1 Administrator Application Architecture

The administrator application shares the project setup, design system, API client and authentication context with the student application built by Govind Jangid. Both are in one repository, and the shared parts live in a common package. Administrator-specific code is organised by feature: club console, event wizard, approvals, finance, scanner and Kanban. Routes under `/admin` are protected by role guards, and each console loads lazily so that the scanner, which must start quickly on a phone, does not download the finance charts.

4.2 Club Admin Console

The dashboard shows upcoming events, registration counts against capacity, recent ledger credits and pending suggestions for the club. A profile page lets the Club Admin update the description, logo, banner and social links of the assigned club only. An executive leads page lists the president, secretary and faculty coordinator. A read-only treasury view shows total revenue, available balance, pending settlement and payout UPI identifier, with a table of ledger entries (type, event, credit, debit, running balance, remarks). There is no edit control on any balance field, matching the rule that club balances change only through backend transactions.

4.3 Event Creation Wizard

The wizard guides a Club Admin through five steps: basic information (title, category, tags, summary, Markdown description, banner); schedule and venue (start, end, venue with immediate conflict check); registration (solo or team, team size limits, capacity, free or paid, ticket price); activity points (category and proposed points); and review and submit. A draft is saved automatically between steps. On submission the status becomes `PENDING_APPROVAL`. Validation is declared in one schema; for example, a paid event must have a positive ticket price, and the end time must be after the start time.

4.4 Venue Availability and Conflict Interface

When a venue and times are chosen the interface calls `POST /venues/check-conflict`. On success it shows the venue name and the 30-minute setup and teardown buffer window.

On the `VENUE_BUFFER_COLLISION` error it shows the conflicting event, the time at which the venue becomes free, and the list of alternative venues as buttons that apply the alternative to the form. A weekly calendar view marks booked slots with their buffers in a lighter colour so that clubs can choose a free slot before submitting.

4.5 Super Admin Approval Queue

The queue lists proposals with club, venue, date, capacity, price and proposed points, sorted by oldest first. Opening a proposal shows the full details and the conflict check result. The Super Admin can approve or reject using a modal; the review note is mandatory for rejection, matching the specification. The decision is sent with `PATCH /events/:id/status`, and the card moves out of the queue. The target in the PRD is to reduce approval turnaround from three business days to under two hours, so the queue also shows how long each proposal has waited.

4.6 Finance and Audit Console

The console shows campus gross revenue, a bar chart of revenue per club, pending settlement, refund status and a filterable transaction table with export. Each row links to the payment identifier and the ledger entry. All views are read-only for the Super Admin. A Club Admin sees only the treasury of the own club. The console refreshes when new payments arrive through the WebSocket channel.

4.7 Gate Scanner Progressive Web App

The scanner is the most time-critical screen. It opens the rear camera, reads QR codes continuously and sends the decoded token with the event identifier to `POST /attendance/verify-scan`. The server validates the signature, time window and event match as described in the specification (Figure 4.1) and answers with an entry decision. The interface turns the whole screen green with the student's name, roll number and ticket number for an approved entry, and red with a loud alarm and the first check-in time for a duplicate.

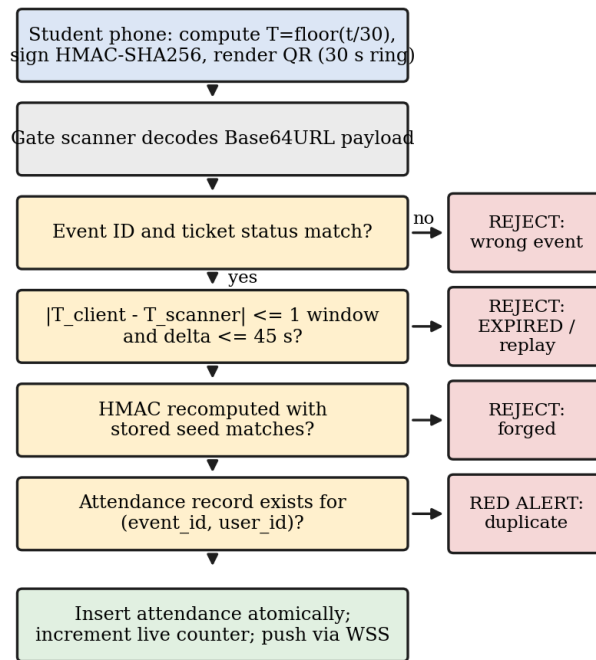


Figure 4.1: Gate-scan validation pipeline (server side) used by the scanner

Table 4.1: Scanner result states and interface behaviour

State	Screen	Sound and extra behaviour
Scanning	Camera view with frame guide, live count	Torch toggle; manual roll-number entry button
Approved	Full green screen, name, roll, ticket number	Short pleasant tone; returns to scanning after 1.5 s
Duplicate (409)	Full red screen, “Already checked in at 09:14 AM”	Loud alarm and vibration; stays until dismissed
Expired token	Orange screen, “Pass expired - ask student to refresh”	Warning tone
Wrong event	Orange screen with the event the pass belongs to	Warning tone
Invalid signature	Red screen, “Invalid pass”	Alarm; event logged with scanner identity
Network error	Grey banner, “Reconnecting”	Scan is queued for retry, not approved offline

To keep the scan fast, the code decodes at a limited frame rate, ignores the same code for two seconds to avoid repeat reads, and sends one request at a time. The target is a validation response under 400 ms on a 4G network. A procedure for unreliable networks is documented: the volunteer can enter the roll number manually, and such entries are flagged in the audit log.

4.8 “You Said, We Did” Kanban Board

Club Admins see four columns: Submitted, Under review, Planned and Implemented. Cards show the suggestion, the number of upvotes and the event. Dragging a card calls the status API. When a card moves to Implemented, a dialog asks for a proof note and optional photograph so that the public board shows what was done. Students see the same board in read-only form.

4.9 Transcript and Sign-Off Screens

The Super Admin can open a student's activity-points summary, view the itemised log and sign off the transcript, which triggers the backend PDF generation. Students and advisors can download the signed transcript. The screen shows the verification QR code that points to the public verification URL.

4.10 Event Copilot Panel

Inside the wizard, a side panel takes a short prompt such as “36-hour AI hackathon for second-year students”. It calls the backend copilot API and shows a draft title, agenda, prerequisites and rules. The Club Admin can accept each part into the form or edit it. The draft is never submitted automatically, and the Super Admin still reviews the final proposal.

4.11 Deployment Architecture

The production stack runs as Docker containers on a college server or cloud virtual machine (Figure 4.2). NGINX is the only public entry point. It serves the built frontend and forwards API and WebSocket requests to the API container. The API connects to MongoDB (a replica set is required for transactions) and optionally to Redis for rate-limit buckets. Razorpay sends webhooks over HTTPS to a public path of NGINX. Secrets are provided through environment variables or a secret store and are never part of the frontend bundle.

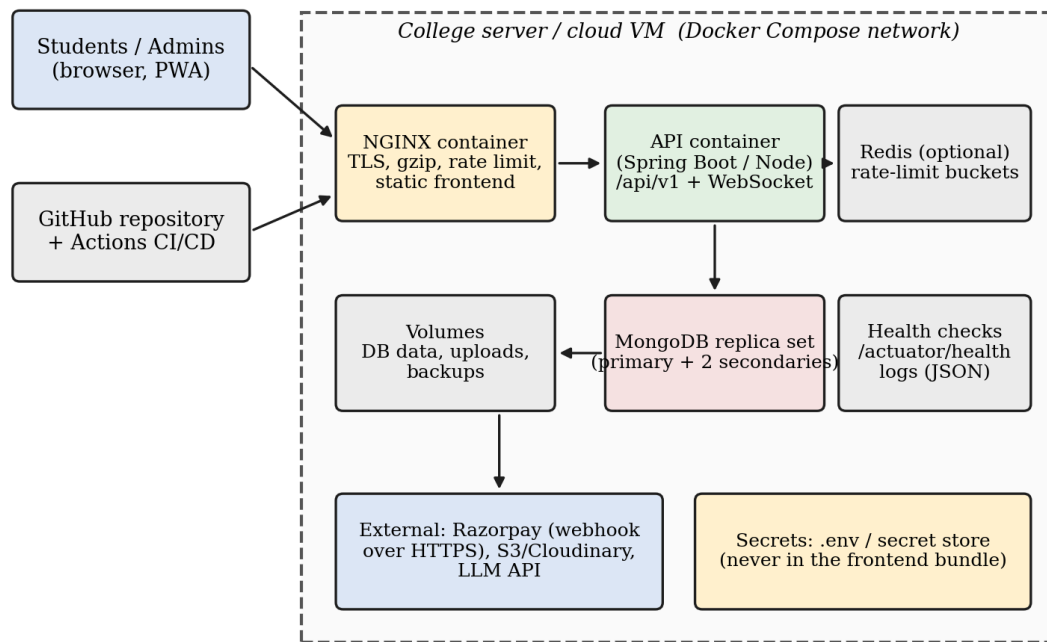


Figure 4.2: Deployment topology of CampusSphere with Docker Compose

4.12 Containers and Compose

```

# frontend/Dockerfile (multi-stage)
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80 443

```

The Compose file defines services for NGINX, the API, MongoDB and Redis on a private network. Only NGINX publishes ports 80 and 443. MongoDB data and backups use named volumes. Each service has a health check, and the API starts only when the database is healthy.

4.13 NGINX, TLS and Rate Limiting

```

limit_req_zone $binary_remote_addr zone=api:10m rate=100r/m;
limit_req_zone $binary_remote_addr zone=auth:10m rate=5r/m;
server {
    listen 443 ssl http2;
    ssl_protocols TLSv1.3;
    location /api/v1/auth/ { limit_req zone=auth burst=3; proxy_pass
http://api:8080; }

```

```

    location /api/ { limit_req zone=api burst=20; proxy_pass
http://api:8080; }
    location /ws/ { proxy_pass http://api:8080; proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade; }
    location / { try_files $uri /index.html; }
}

```

Static assets are served with long cache headers because file names contain a content hash, while `index.html` is not cached so that a new release is picked up immediately. Security headers (Content-Security-Policy, X-Content-Type-Options, Referrer-Policy) are added. The CORS configuration allows only the configured Arya College domains.

4.14 CI/CD Pipeline

Table 4.2: CI/CD pipeline stages

Stage	Actions	Gate
Pull request	Install, lint, type check, unit tests, build, container build	All must pass; one review required
Merge to main	Build and push images with commit tag	Image scan for known vulnerabilities
Staging deploy	Compose pull and up on staging server; smoke tests	Health checks and smoke tests pass
Production deploy	Manual approval, then rolling restart	Health checks pass; rollback if not
After deploy	Notify team, record version, run synthetic check	Alert on failure

4.15 Environment, Secrets and Backups

Settings are separated into development, staging and production files. Secrets such as the JWT signing key, Razorpay secret, webhook secret and database password are stored outside the repository and injected at start. Only public values such as the Razorpay key identifier reach the browser. Daily MongoDB backups are written to a separate volume and copied off the server; a restore drill is run in Week 12 with Anshika Pathak to prove that a backup can actually be restored.

4.16 Monitoring, Rollback and Runbook

Health endpoints for the API and database are polled; structured logs carry a trace identifier so that a request can be followed across containers. A runbook lists how to deploy, roll back to the previous image tag, rotate secrets, restore a backup and what to check when payments or scanning fail during a festival. During large events, the team follows a checklist before the gates open.

4.17 Testing Approach

Table 4.3: Test plan for my section

Level	Tool	Examples
Unit	Vitest	Wizard schema rules, buffer message formatting, scanner state machine
Component	React Testing Library	Approval modal requires note on reject; Kanban drop calls API
Device	Physical Android and iOS phones	Camera permission, torch, QR read speed, alarm volume
End to end	Playwright	Create event, approve, register, scan, duplicate scan
Load	k6	Flash registration of 500 users; 20 scans per second at the gate
Deployment	Smoke tests, restore drill	Fresh server up in one command; backup restore works

4.18 Anticipated Challenges and Planned Solutions

Table 4.4: Challenges and solutions

Challenge	Planned solution
Camera needs HTTPS and permission, behaves differently on each phone	Test on several devices early; guide screens for permission; manual entry fallback
Volunteers need instant feedback in bright light	Full-screen colours, large text, sound and vibration
Weak venue network	Lightweight requests, clear offline state, queued retry, documented manual procedure
Admin screens drift from the student UI	Shared design system and API types in a common package
Risky releases before a festival	Staging first, smoke tests, rollback by image tag, release freeze before big events
Secrets accidentally committed	Secret scanning in CI, environment injection, rotation procedure

4.19 Integration with Teammates

I rely on the APIs of Garvita Jain (events, venues, approvals, audit), Ayushi Garg (registration roster, payments, ledger), and Anshika Pathak (verify-scan, feedback, Kanban, points) and share the application shell with Govind Jangid. I start each screen against mock responses that follow the API specification and switch to the real endpoints when they are available. In Week 12 I provide images and environment variable lists to all members and collect their service requirements.

CHAPTER 5

14-WEEK WORK PLAN

5.1 My Week-by-Week Plan

Table 5.1 gives my plan for the 14 weeks together with the expected deliverable of each week. The weeks follow the ten phases of the roadmap.

Table 5.1: My 14-week plan

Week	Phase	My tasks	Deliverable / check
1	Setup	Git branching, ESLint/Prettier, Docker dev compose, CI skeleton	CI runs on a sample pull request
2	1	Admin app shell; route guards; shared API client; environment configs	Admin shell opens with role guards
3	2	Club admin console: profile edit and executive leads	Club admin edits only own club
4	3	Event creation wizard (title, category, mode, price, capacity, points)	Wizard saves a draft with validation
5	3	Super Admin approval queue; approve/reject modal	Approve and reject update the queue
6	4	Venue conflict alert UI and venue calendar	Conflict alert shows alternatives
7	5	Super Admin finance and audit console UI	Totals match ledger entries
8	6	Gate scanner PWA: camera QR, green/red banners, alarm, live counter	Valid scan green; duplicate red with alarm
9	7	Kanban board UI for Club Admin	Card moves between columns and saves
10	8	Transcript download UI; Super Admin sign-off screen	Signed transcript downloads
11	9	Event Copilot panel inside the wizard	Copilot draft fills wizard fields
12	10	Dockerfiles, compose, NGINX, TLS, secrets	Stack starts with one command
13	Test	CI/CD pipeline; staging deploy; load tests	Pipeline deploys to staging; load test report
14	Pilot	Production deployment; runbook	Production live; runbook handed over

5.2 Team Overview and Dependencies

Figure 5.1 shows the whole team schedule. In every week each member has one main deliverable, and the weekly meeting reviews all five. The dependencies between my tasks and the work of the other members are listed in Table 5.2.

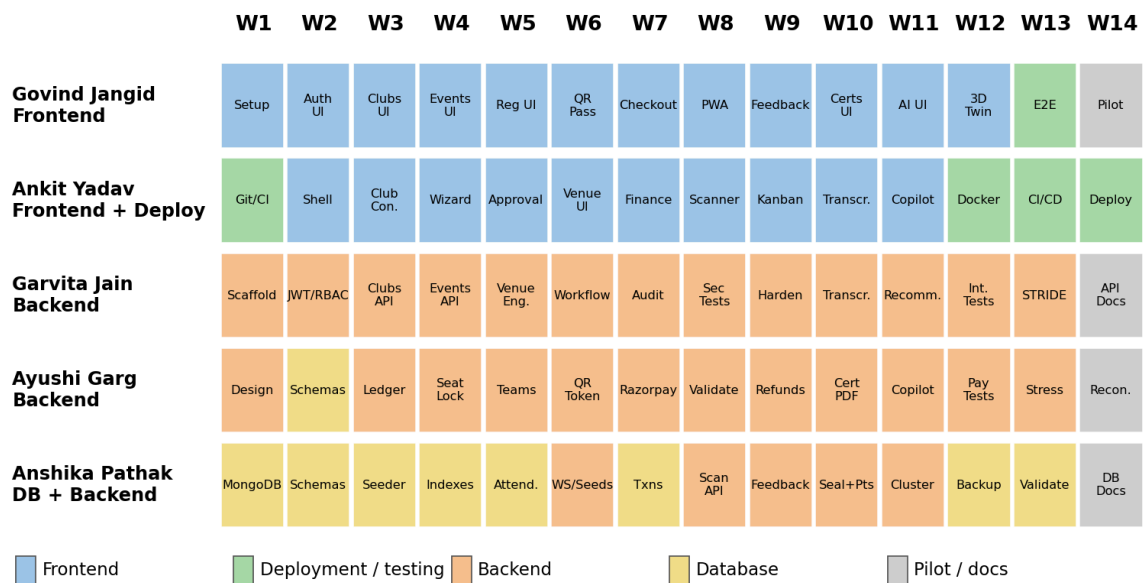


Figure 5.1: 14-week work distribution of the five members

Table 5.2: Dependencies of my weekly tasks on teammates

Week	My task	Depends on / provides to
2	Admin shell and guards	Garvita Jain: role claims; Govind Jangid: shared API client
3	Club console	Garvita Jain: clubs API; Anshika Pathak: seeded clubs
4-5	Wizard and approvals	Garvita Jain: events API and status workflow
6	Venue conflict UI	Garvita Jain: venue buffer engine
7	Finance console	Ayushi Garg: ledger; Garvita Jain: audit APIs
8	Gate scanner	Anshika Pathak: verify-scan API; Ayushi Garg: token format
9	Kanban	Anshika Pathak: suggestion and status APIs
10	Transcript	Garvita Jain: transcript API; Anshika Pathak: points
12-14	Deployment	All members: images, environment variables and health endpoints

5.3 Reporting and Review

Every Friday I give the Team Leader a short status note listing completed tasks, tasks in progress, blockers and changes to interfaces. The guide reviews progress at the end of key milestones (Weeks 2, 5, 7, 9, 11, 13 and 14). A slip is first handled by re-ordering tasks within the week and then, if needed, by moving work between sections in agreement with the affected members.

CHAPTER 6

LEARNING OUTCOMES AND SKILLS

6.1 Technical Skills

- Building role-based administrative consoles with typed forms, tables and charts.
- Using browser camera, audio and vibration APIs to build a fast scanning application.
- Designing optimistic user interfaces with drag and drop and server reconciliation.
- Writing multi-stage Dockerfiles and Compose stacks with health checks.
- Configuring NGINX for TLS, compression, caching and rate limiting.
- Building CI/CD pipelines with GitHub Actions and rollback procedures.
- Load testing with k6 and using logs and health checks for observability.

6.2 Professional Skills

- Reading and applying a formal specification suite (PRD, SRS, TRD, ERD, API specification).
- Contract-first collaboration between the frontend, backend and database sections.
- Planning work in weekly deliverables and tracking progress with an issue board.
- Writing clear documentation and status reports.

6.3 Teamwork

Deployment touches the work of every member, so I learned the importance of agreeing early on environment variables, health endpoints and start-up order. I also learned to give the backend members quick feedback when a response format makes the interface awkward, and to accept the same from them.

6.4 Problem Solving and Debugging

Operational problems usually appear only under realistic conditions, such as a phone camera in sunlight or many scans at once. I plan to reproduce each such problem with a test device or load script, isolate the layer responsible and add an automated check once it is fixed.

6.5 Version Control and Communication

Short-lived branches, small pull requests and meaningful commit messages make collaboration smoother and keep the main branch always deployable. Clear and timely

communication with the other four members, including early notice of changes to an interface, is as important as writing good code.

CHAPTER 7

CONCLUSION

This individual report presented my part in the CampusSphere project: the administrator frontend, the gate scanner and the deployment of the system. The work is divided among five members into two frontend and deployment sections and three backend and database sections, planned over 14 weeks following the ten-phase roadmap.

My modules give organisers and the Dean the tools they need: a guided wizard for events, a clear approval queue, a read-only and auditable view of club money, a scanner that gives an instant and unmistakable decision, and a feedback board that shows students what was done. The deployment design makes the whole platform reproducible, secure over HTTPS and recoverable through backups and rollback.

This report documents the design and the plan of my work. Performance and reliability figures are targets to be measured in Weeks 12 to 14 and in the pilot. The experience has given me a solid foundation in frontend engineering for operational tools and in DevOps practices, which I will use in future projects and in professional work.

REFERENCES

- [1]AICTE, “Mandatory extracurricular activity points for B.Tech degree,” All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota.
- [2]CampusSphere Project Team, “Product Requirements Document (PRD), v1.0.0-PROD,” Arya College of Engineering & I.T., Jaipur, 2026.
- [3]CampusSphere Project Team, “Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD,” 2026.
- [4]CampusSphere Project Team, “Technical Requirements Document (TRD), v1.0.0-PROD,” 2026.
- [5]CampusSphere Project Team, “REST API Specification, v1.0.0-PROD,” 2026.
- [6]CampusSphere Project Team, “Security Architecture and Fraud Prevention Specification, v1.0.0-PROD,” 2026.
- [7]CampusSphere Project Team, “Engineering Development Roadmap, v1.0.0-PROD,” 2026.
- [8]Meta Platforms, “React documentation.” [Online]. Available: <https://react.dev/>
- [9]React Hook Form, “Documentation.” [Online]. Available: <https://react-hook-form.com/>
- [10]Mozilla Developer Network, “MediaDevices.getUserMedia() and BarcodeDetector API.” [Online]. Available: <https://developer.mozilla.org/>
- [11]Mozilla Developer Network, “Progressive web apps.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- [12]Docker Inc., “Docker and Docker Compose documentation.” [Online]. Available: <https://docs.docker.com/>
- [13]F5 NGINX, “NGINX documentation.” [Online]. Available: <https://nginx.org/en/docs/>
- [14]GitHub, “GitHub Actions documentation.” [Online]. Available: <https://docs.github.com/actions>
- [15]Grafana Labs, “k6 load testing documentation.” [Online]. Available: <https://grafana.com/docs/k6/>
- [16]Playwright Contributors, “Playwright documentation.” [Online]. Available: <https://playwright.dev/>
- [17]Razorpay, “Payment gateway and webhook documentation.” [Online]. Available: <https://razorpay.com/docs/>